```
static struct spi_board_info pch_spi_slaves[] = {
     {
      .modalias = "m25p80",     /* Name of spi_driver for
this device*/
      .max_speed_hz = 33000000, /* max spi clock (SCK) speed
in HZ*/
      .bus_num = 0,             /* Framework bus number*/
      .chip_select = 0,         /* Framework chip select.*/
      .platform_data = &pch_spi_flash,
      .mode = SPI_MODE_0,
     },
};

spi_register_board_info (pch_spi_slaves, ARRAY_SIZE(pch_spi_
slaves));
```

This mapping driver registers flash chip information with the SPI framework. The *modalias* string must be set to the name of the chip driver going to control the flash. You can verify that *m25p80.c* registers itself with the same name. The *max_speed_hz* is a flash chip parameter and must be obtained from the hardware manual. The *bus_number* is the SPI bus number. The *chipselect* can be 0 or 1. The *chipselect* is used by a SPI *master* to control two SPI slaves. This information is specific to the board architecture. The *platform_data* will be used by the chip driver and the MTD sub-system for chip identification and partitioning.

The *spi_register_board_info()* function stores the flash information in a global link list pointed to by *board_list*. Later, this list is scanned by *spi_match_master_to_boardinfo()* to check if any of the slaves can be claimed by the master.

SPI master registration to the SPI framework

The PCI driver takes care of identifying the PCH and registering the SPI master. Remember, the SPI controller is a part of PCH; therefore, the PCI driver has to identify its PCH first. Readers can refer to my article *http://linuxgazette.net/156/jangir.html* to learn about PCI device identification and initialisation. This article assumes that pDev is pointing to the PCH PCI device.

Register the SPI master through the PCI driver:

```
/* allocate SPI master */
struct spi_master *pmaster = spi_alloc_master(&pDev->dev,
sizeof(struct drv_private_data));

/* initialize SPI master */
pmaster->bus_num = 0;
pmaster->num_chipselect = 0;
pmaster->setup = pch_spi_setup;
pmaster->transfer = pch_spi_transfer;
pmaster->cleanup = pch_spi_cleanup;

/* initialize driver private data */
struct drv_private_data *priv = spi_master_get_
devdata(pmaster);


/*Register the controller with the SPI core. * /
spi_register_master(pmaster);
```

The code snippet shown above registers an SPI master with the SPI framework. This SPI master represented by *struct spi_master* stands for the SPI master controller on the PCH. This SPI master has the responsibility of managing SPI slave devices like SPI flash.

Every SPI master has to register three methods with the SPI framework. These methods are setup, transfer and cleanup. Each of these methods is called when an SPI slave device is created, operated and removed.

SPI slave device creation by SPI framework

After the board driver and PCI driver have registered flash information (SPI slave device information) and have created and registered the SPI master, the SPI framework creates slave SPI devices. The slave SPI devices are represented by *struct spi_device*.

During *spi_register_master* execution, the SPI framework checks if there is any SPI slave device that is hooked to the same bus number registered by the SPI master. If such a device is found, the SPI framework creates slave SPI devices and registers those devices with the device subsystem. This is done by a call to *spi_match_master_to_boardinfo()* function (from *spi_register_master* function).

SPI slave devices can be created by the SPI framework in the following way:

```
void spi_match_master_to_boardinfo(struct spi_master
*master, struct spi_board_info *bi)
{
     struct spi_device *dev;

     if (master->bus_num != bi->bus_num)
       return;

     dev = spi_new_device(master, bi);
}
```

Registration to the MTD sub-system

Now that the SPI slave device is registered with the device sub-system, an MTD device is registered with the MTD sub-system so that the flash can be accessed through the MTD sub-system like */dev/mtd, /dev/mtdblock*. These MTD devices are used by MTD utilities like *flashcp, erase_all*, etc. But this registration is done neither by the board driver nor by the PCI driver. This is done by the chip driver (m25p80.c). The probe function of the chip driver is invoked by the SPI framework whenever it creates an SPI slave device. The *modalias* field